1. Problem Statement

Credit risk management is an important part of the banking and financial sector. Financial institutions provide loans to customers based on factors such as income, employment history, credit score, loan amount, and repayment capacity. However, some borrowers fail to repay their loans, resulting in financial losses.

Traditional loan assessment methods can be time-consuming and may not effectively identify complex relationships between financial attributes. Therefore, machine learning techniques can be used to analyze historical loan information and identify patterns associated with loan defaults.

This project aims to develop a loan default prediction system using Apache Spark MLlib. The system analyzes applicant information, processes the dataset, and applies Logistic Regression and Random Forest classification algorithms to predict whether a loan is likely to default.

Apache Spark and PySpark are used for data processing, while Pandas and Matplotlib are used for data analysis and visualization. The project demonstrates how Spark-based machine learning can support financial risk assessment.

2. Objectives of the Project

The main objective of this project, "Credit Risk Analysis and Loan Default Prediction Using Apache Spark MLlib," is to develop a machine learning-based system that analyzes loan applicant information and predicts the possibility of loan default using Apache Spark and PySpark.

The project focuses on applying cluster computing concepts, data preprocessing, machine learning algorithms, and data visualization techniques to a real-world financial dataset.

Specific Objectives

1. To implement Apache Spark for credit risk analysis

To use Apache Spark as the primary data-processing framework and PySpark as the programming interface for handling and analyzing the credit risk dataset efficiently.

2. To perform data preprocessing and cleaning

To improve data quality by removing duplicate records, handling missing values, converting categorical attributes into numerical representations, and preparing the dataset for machine learning.

3. To analyze loan applicant information

To examine important applicant and loan characteristics, including age, annual income, employment length, loan amount, loan purpose, loan grade, interest rate, and credit history.

4. To visualize credit risk patterns

To generate five visualizations using Matplotlib to understand the loan default distribution, loan purposes, loan grades, applicant income distribution, and model performance.

5. To implement machine learning algorithms using Spark MLlib

To develop two classification models:

- Logistic Regression: To predict loan default status using a statistical classification approach.
- Random Forest: To predict loan default status using an ensemble of decision trees.

6. To evaluate and compare model performance

To evaluate both machine learning models using Accuracy, F1 Score, and ROC-AUC and compare their performance in predicting loan default status.

7. To demonstrate Spark-based data processing

To use Spark DataFrames, feature transformation techniques, and Spark MLlib pipelines for efficient data processing and machine learning. The implementation runs in Spark's local multi-core mode.

8. To generate and store prediction results

To save the Random Forest model's prediction results in CSV format for further analysis and interpretation.

3. Dataset Description

The dataset used in this project is the Credit Risk Dataset, which contains information about loan applicants, their financial conditions, employment details, credit history, and loan characteristics. The dataset is used to analyze credit risk and predict whether an applicant is likely to default on a loan.

The dataset is suitable for implementing machine learning classification algorithms because it contains both numerical and categorical attributes. These attributes help identify patterns and relationships between applicant characteristics and loan default status.

In this project, the dataset is processed using Apache Spark DataFrames and PySpark, while Spark MLlib is used to train Logistic Regression and Random Forest classification models.

3.1 Dataset Overview

Property	Description
Dataset Name	Credit Risk Dataset
File Name	credit_risk_dataset.csv
Dataset Format	CSV (Comma-Separated Values)
Total Records	32,581
Total Attributes	12
Numerical Attributes	8, including the target variable
Categorical Attributes	4
Target Variable	loan_status
Machine Learning Task	Binary Classification
Processing Framework	Apache Spark (PySpark)
Machine Learning Library	Spark MLlib

The original dataset contains **32,581 records and 12 attributes**. It includes both numerical and categorical variables that help identify relationships between applicant information and loan default status.

3.2 Dataset Attributes

S. No.	Attribute Name	Description
1	person_age	Age of the loan applicant in years
2	person_income	Annual income of the applicant
3	person_home_ownership	Home ownership status, such as RENT, OWN, or MORTGAGE
4	person_emp_length	Employment experience in years
5	loan_intent	Purpose of the loan, such as education, medical, or personal expenses
6	loan_grade	Credit grade assigned to the loan
7	loan_amnt	Amount of the loan

8	loan_int_rate	Interest rate associated with the loan
9	loan_status	Loan default status (0 or 1)
10	loan_percent_income	Ratio of loan amount to applicant income
11	cb_person_default_on_file	Indicates whether the applicant has a previous default record
12	cb_person_cred_hist_length	Length of the applicant's credit history in years

3.3 Target Variable

The target variable for this project is loan_status, which represents whether a loan is classified as defaulted or non-defaulted.

Value	Classification	Meaning
0	Non-Default	The loan is not classified as defaulted
1	Default	The loan is classified as defaulted

The machine learning models analyze the remaining 11 attributes to predict the value of loan_status

3.4 Loan Status Distribution

The original dataset contains the following loan-status distribution:

Loan Status	Number of Records	Percentage
Non-Default (0)	25,473	78.18%
Default (1)	7,108	21.82%
Total	32,581	100%

3.5 Data Quality and Preprocessing

The original dataset contains some missing values and duplicate records that must be addressed before model training.

The following data quality issues were identified:

- Missing employment length: 895 records.
- Missing loan interest rate: 3,116 records.

- Duplicate records: 165 records.

The Python program performs data preprocessing using Apache Spark. Duplicate records are removed using dropDuplicates(), missing categorical values are replaced with Unknown, and numerical missing values are handled using Spark MLlib's median-based Imputer.

Categorical features are transformed using StringIndexer and OneHotEncoder, while VectorAssembler combines the processed features into a single vector for machine learning.

4. Dataset Source

The dataset used in this project, "Credit Risk Analysis and Loan Default Prediction Using Apache Spark MLlib," is obtained from Kaggle, an online platform that provides datasets for data science, machine learning, and analytics projects.

The selected dataset is named Credit Risk Dataset and contains information about loan applicants, including their income, employment history, home ownership, loan amount, interest rate, loan purpose, credit history, and loan default status.

4.1 Dataset Source

Dataset Link: https://www.kaggle.com/datasets/laotse/credit-risk-dataset?utm_source

4.2 Reason for Selecting the Dataset

The Credit Risk Dataset was selected because it contains important financial and personal attributes that can help identify loan-default patterns.

The dataset is suitable for this project for the following reasons:

1. Real-world application: It represents a practical financial classification problem related to credit risk assessment.
2. Multiple attributes: It contains both numerical and categorical variables, making it suitable for feature engineering and machine learning.
3. Binary classification: The loan_status attribute allows the implementation of classification algorithms such as Logistic Regression and Random Forest.
4. Apache Spark compatibility: The CSV dataset can be loaded and processed using PySpark DataFrames and Spark MLlib pipelines.

5. Data visualization: The dataset supports graphical analysis of loan status, loan purpose, loan grade, and applicant income.

5. Tools & Technologies Used

1. Apache Spark

Apache Spark is an open-source distributed computing framework used for large-scale data processing. In this project, Spark handles dataset loading, cleaning, and transformation. The application runs in local multi-core mode.

2. Python

Python is the main programming language used to implement the project, including data processing, model training, evaluation, and visualization.

3. PySpark

PySpark provides the Python interface for Apache Spark. It is used to perform DataFrame operations and implement machine learning pipelines.

4. Spark SQL and DataFrames

Spark DataFrames are used to load the CSV dataset, remove duplicate records, group data, and perform transformations.

5. Spark MLlib

Spark MLlib provides machine learning algorithms and preprocessing tools. Logistic Regression and Random Forest are implemented using the pyspark.ml API.

6. Pandas

Pandas is used to organize aggregated data and prepare model evaluation results for comparison.

7. Matplotlib

Matplotlib is used to generate five charts for exploratory data analysis and model performance comparison.

8. Java Development Kit

Java provides the runtime environment required for Apache Spark execution.

9. Windows Command Prompt

Windows CMD is used to execute the Python program and display the evaluation results.

6. Implementation / Methodology

The project follows a systematic approach to analyze credit risk and predict loan defaults using Apache Spark MLlib.

Step 1: Data Collection

The Credit Risk Dataset is collected from Kaggle in CSV format. It contains 32,581 records and 12 attributes related to loan applicants.

Step 2: Data Loading and Preprocessing

The dataset is loaded using PySpark DataFrames. Duplicate records are removed, missing values are handled, and the target variable loan_status is converted into numerical format.

Step 3: Exploratory Data Analysis

Spark DataFrames, Pandas, and Matplotlib are used to analyze the dataset. Four charts are generated to visualize loan status, loan purpose, loan grade, and applicant income distribution.

Step 4: Feature Engineering

Spark MLlib's Imputer, StringIndexer, OneHotEncoder, and VectorAssembler are used to transform numerical and categorical attributes into machine learning feature vectors.

Step 5: Model Training

The dataset is divided into 80% training and 20% testing data. Logistic Regression and Random Forest models are trained using Spark MLlib pipelines.

Step 6: Model Evaluation

Both models are evaluated using Accuracy, F1 Score, and ROC-AUC to measure their classification performance.

Step 7: Result Visualization and Storage

A fifth chart compares model performance. Random Forest predictions are saved in CSV format for further analysis.

Methodology Workflow:

Data Collection → Data Loading → Preprocessing → Visualization → Feature Engineering → Model Training → Evaluation → Result Analysis

7. Complete Source Code

```
import warnings
warnings.filterwarnings("ignore")  # Suppresses standard Python warnings
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    Imputer, StringIndexer,
    OneHotEncoder, VectorAssembler
)
from pyspark.ml.classification import (
    LogisticRegression,
    RandomForestClassifier
)
from pyspark.ml.evaluation import (
    MulticlassClassificationEvaluator,
    BinaryClassificationEvaluator
)
import pandas as pd
import matplotlib.pyplot as plt
import os

os.makedirs("outputs", exist_ok=True)

# Start Spark
spark = SparkSession.builder \
    .appName("Credit Risk Analysis") \
    .master("local[*]") \
    .getOrCreate()

# Suppresses all PySpark Java/DAGScheduler warnings (Forces it to only show Errors)
spark.sparkContext.setLogLevel("ERROR")

# Load Dataset
df = spark.read.csv(
    "credit_risk_dataset.csv",
```

```
    header=True,
    inferSchema=True
)
df = df.dropDuplicates()
df = df.withColumn(
    "loan_status",
    col("loan_status").cast("double")
)
df = df.dropna(subset=["loan_status"])
df = df.fillna("Unknown", subset=[
    "person_home_ownership",
    "loan_intent",
    "loan_grade",
    "cb_person_default_on_file"
])

print("Dataset Schema:")
df.printSchema()
print("Total Records:", df.count())

# Chart 1: Loan Default Distribution
status = df.groupBy("loan_status").count().toPandas()
status["loan_status"] = status["loan_status"].map(
    {0.0: "No Default", 1.0: "Default"}
)
plt.figure(figsize=(7, 5))
plt.bar(status["loan_status"], status["count"])
plt.title("Loan Default Distribution")
plt.xlabel("Loan Status")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.savefig("outputs/chart1_loan_status.png")
plt.close()
```

```
# Chart 2: Loan Purpose
intent = df.groupBy("loan_intent").count().toPandas()
plt.figure(figsize=(8, 5))
plt.bar(intent["loan_intent"], intent["count"])
plt.title("Loan Applications by Purpose")
plt.xlabel("Loan Purpose")
plt.ylabel("Applications")
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig("outputs/chart2_loan_purpose.png")
plt.close()

# Chart 3: Loan Grade
grade = df.groupBy("loan_grade").count().toPandas()
grade = grade.sort_values("loan_grade")
plt.figure(figsize=(7, 5))
plt.bar(grade["loan_grade"], grade["count"])
plt.title("Loan Grade Distribution")
plt.xlabel("Loan Grade")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.savefig("outputs/chart3_loan_grade.png")
plt.close()

# Chart 4: Applicant Income
income = df.select("person_income").dropna().toPandas()
plt.figure(figsize=(7, 5))
plt.hist(income["person_income"], bins=30)
plt.title("Applicant Income Distribution")
plt.xlabel("Annual Income")
plt.ylabel("Frequency")
plt.tight_layout()
```

```
plt.savefig("outputs/chart4_income.png")
plt.close()

# Feature Selection
numeric_cols = [
    "person_age",
    "person_income",
    "person_emp_length",
    "loan_amnt",
    "loan_int_rate",
    "loan_percent_income",
    "cb_person_cred_hist_length"
]

categorical_cols = [
    "person_home_ownership",
    "loan_intent",
    "loan_grade",
    "cb_person_default_on_file"
]

imputer = Imputer(
    inputCols=numeric_cols,
    outputCols=[c + "_imputed" for c in numeric_cols],
    strategy="median"
)

indexer = StringIndexer(
    inputCols=categorical_cols,
    outputCols=[c + "_index" for c in categorical_cols],
    handleInvalid="keep"
)
```

```
encoder = OneHotEncoder(
    inputCols=[c + "_index" for c in categorical_cols],
    outputCols=[c + "_encoded" for c in categorical_cols],
    handleInvalid="keep"
)

assembler = VectorAssembler(
    inputCols=[c + "_imputed" for c in numeric_cols] +
              [c + "_encoded" for c in categorical_cols],
    outputCol="features"
)

# Train-Test Split
train, test = df.randomSplit([0.8, 0.2], seed=42)

# Classification Models
lr = LogisticRegression(
    featuresCol="features",
    labelCol="loan_status",
    maxIter=50
)

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="loan_status",
    numTrees=100,
    maxDepth=8,
    seed=42
)

# Train Models
lr_model = Pipeline(
    stages=[imputer, indexer, encoder, assembler, lr]
```

```
).fit(train)

rf_model = Pipeline(
    stages=[imputer, indexer, encoder, assembler, rf]
).fit(train)

lr_predictions = lr_model.transform(test)
rf_predictions = rf_model.transform(test)

# Evaluation
accuracy_eval = MulticlassClassificationEvaluator(
    labelCol="loan_status",
    predictionCol="prediction",
    metricName="accuracy"
)

f1_eval = MulticlassClassificationEvaluator(
    labelCol="loan_status",
    predictionCol="prediction",
    metricName="f1"
)

auc_eval = BinaryClassificationEvaluator(
    labelCol="loan_status",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

results = []

for name, predictions in [
    ("Logistic Regression", lr_predictions),
    ("Random Forest", rf_predictions)
```

```
]:
    accuracy = accuracy_eval.evaluate(predictions)
    f1 = f1_eval.evaluate(predictions)
    auc = auc_eval.evaluate(predictions)

    print("\nModel:", name)
    print("Accuracy:", round(accuracy, 4))
    print("F1 Score:", round(f1, 4))
    print("ROC-AUC:", round(auc, 4))

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "F1 Score": f1,
        "ROC-AUC": auc
    })

# Chart 5: Model Comparison
results_df = pd.DataFrame(results).set_index("Model")
results_df.plot(kind="bar", figsize=(9, 5))
plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("outputs/chart5_model_comparison.png")
plt.close()

# Save Predictions
rf_predictions.select(
    "loan_status",
    "prediction",
    col("probability").cast("string").alias("probability")
```

```
).write.mode("overwrite").option(
    "header", True
).csv("outputs/loan_predictions")

# Save Models
lr_model.write().overwrite().save("outputs/logistic_model")
rf_model.write().overwrite().save("outputs/random_forest_model")

spark.stop()
```

8. Execution Screenshots

Screenshot 1: Program Execution

```
C:\Users\tunga\OneDrive\Documents\Projects\Credit-Risk-Analysis>py credit_risk.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

DATASET SHAPE
Rows: 32581
Columns: 12

DATASET COLUMNS
['person_age', 'person_income', 'person_home_ownership', 'person_emp_length', 'loan_intent', 'loan_grade', 'loan_amnt', 'loan_int_rate', 'loan_s
tatus', 'loan_percent_income', 'cb_person_default_on_file', 'cb_person_cred_hist_length']

FIRST FIVE ROWS
+----------+-------------+--------------------+-----------------+-----------+----------+---------+-------------+-----------+------------------
+--------------------------+--------------------------+
|person_age|person_income|person_home_ownership|person_emp_length|loan_intent|loan_grade|loan_amnt|loan_int_rate|loan_status|loan_percent_income
|cb_person_default_on_file|cb_person_cred_hist_length|
+----------+-------------+--------------------+-----------------+-----------+----------+---------+-------------+-----------+------------------
+--------------------------+--------------------------+
|22        |59000        |RENT                |123.0            |PERSONAL   |D         |35000    |16.02        |1          |0.59
|Y                         |3                         |
|21        |9600         |OWN                 |5.0              |EDUCATION  |B         |1000     |11.14        |0          |0.1
|N                         |2                         |
|25        |9600         |MORTGAGE            |1.0              |MEDICAL    |C         |5500     |12.87        |1          |0.57
|N                         |3                         |
|23        |65500        |RENT                |4.0              |MEDICAL    |C         |35000    |15.23        |1          |0.53
|N                         |2                         |
|24        |54400        |RENT                |8.0              |MEDICAL    |C         |35000    |14.27        |1          |0.55
|Y                         |4                         |
+----------+-------------+--------------------+-----------------+-----------+----------+---------+-------------+-----------+------------------
+--------------------------+--------------------------+
only showing top 5 rows
```

Screenshot 2: Dataset Schema and Records

```
DATASET SCHEMA
root
 |-- person_age: integer (nullable = true)
 |-- person_income: integer (nullable = true)
 |-- person_home_ownership: string (nullable = true)
 |-- person_emp_length: double (nullable = true)
 |-- loan_intent: string (nullable = true)
 |-- loan_grade: string (nullable = true)
 |-- loan_amnt: integer (nullable = true)
 |-- loan_int_rate: double (nullable = true)
 |-- loan_status: integer (nullable = true)
 |-- loan_percent_income: double (nullable = true)
 |-- cb_person_default_on_file: string (nullable = true)
 |-- cb_person_cred_hist_length: integer (nullable = true)


MISSING VALUES
+----------+-------------+---------------------+-----------------+-----------+----------+---------+-------------+-----------+------
------------+-----------------------+-------------------------+
|person_age|person_income|person_home_ownership|person_emp_length|loan_intent|loan_grade|loan_amnt|loan_int_rate|loan_status|loan_p
ercent_income|cb_person_default_on_file|cb_person_cred_hist_length|
+----------+-------------+---------------------+-----------------+-----------+----------+---------+-------------+-----------+------
------------+-----------------------+-------------------------+
|0         |0            |0                    |895              |0          |0         |0        |3116         |0          |0
            |0                      |0                         |
+----------+-------------+---------------------+-----------------+-----------+----------+---------+-------------+-----------+------
------------+-----------------------+-------------------------+
```

Screenshot 3: Machine Learning Evaluation

```
C:\Users\tunga\OneDrive\Documents\Projects\Credit-Risk-Analysis>py credit_risk.py
 /C:"Logistic Regression" /C:"Random Forest"
MODEL PERFORMANCE COMPARISON
0  Logistic Regression    0.8694     0.8619  0.8694    0.8623   0.8752
1        Random Forest    0.9342     0.9363  0.9342    0.9302   0.9216
```

9. Output / Result Screenshots

Chart 1: Loan Default Distribution

This chart displays the number of default and non-default loan records. It helps identify the distribution of the target variable.

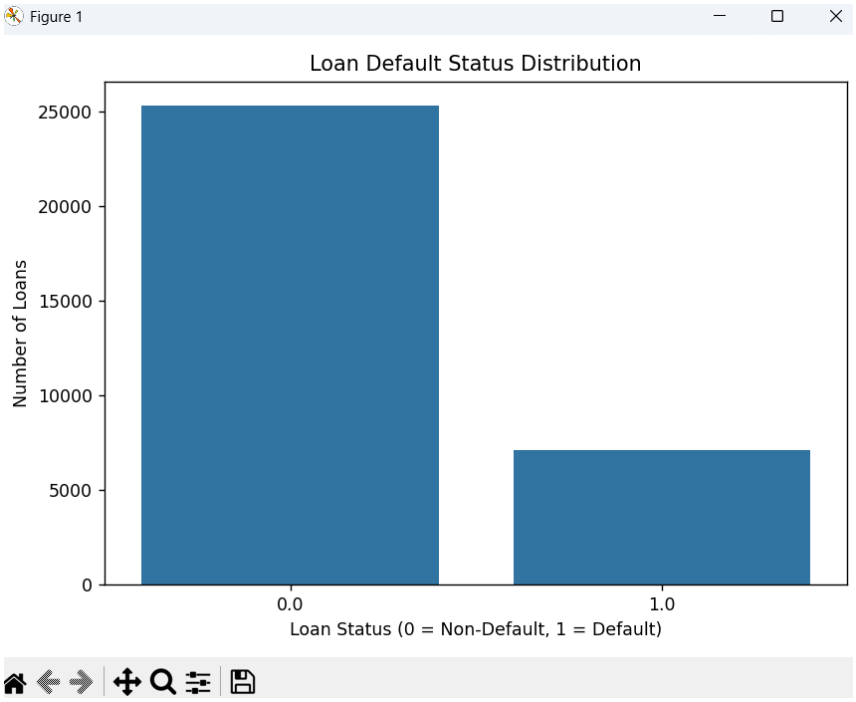


Chart 2: Missing Values by Column

This chart displays the missing values in different dataset columns. The loan interest rate and employment length columns contain missing values. These are handled during data preprocessing.

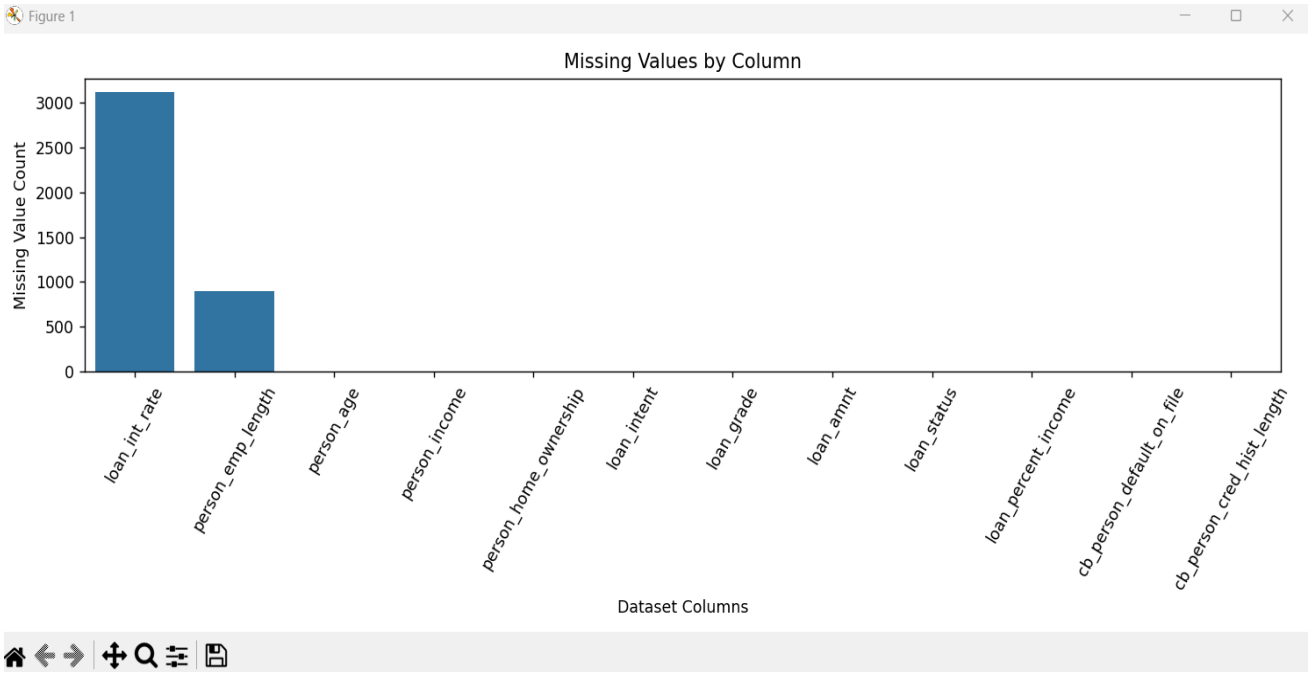


Chart 3: Credit Risk Correlation Heatmap

The correlation heatmap shows the relationships between numerical features such as income, loan amount, interest rate, and loan status. It helps identify important factors related to loan default.

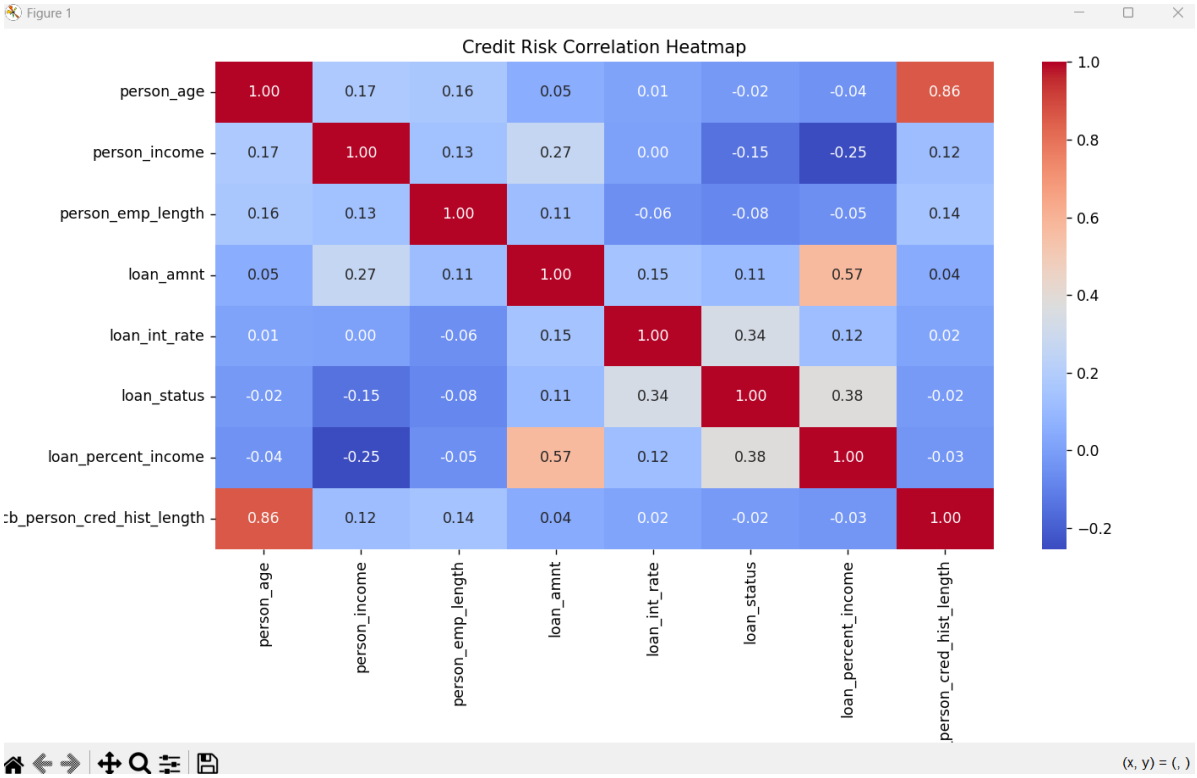


Chart 4: Model Performance Comparison

This chart compares Logistic Regression and Random Forest models using accuracy, precision, recall, F1-score, and ROC-AUC. Random Forest achieved 93.42% accuracy, while Logistic Regression achieved 86.94% accuracy. Random Forest performed better overall.

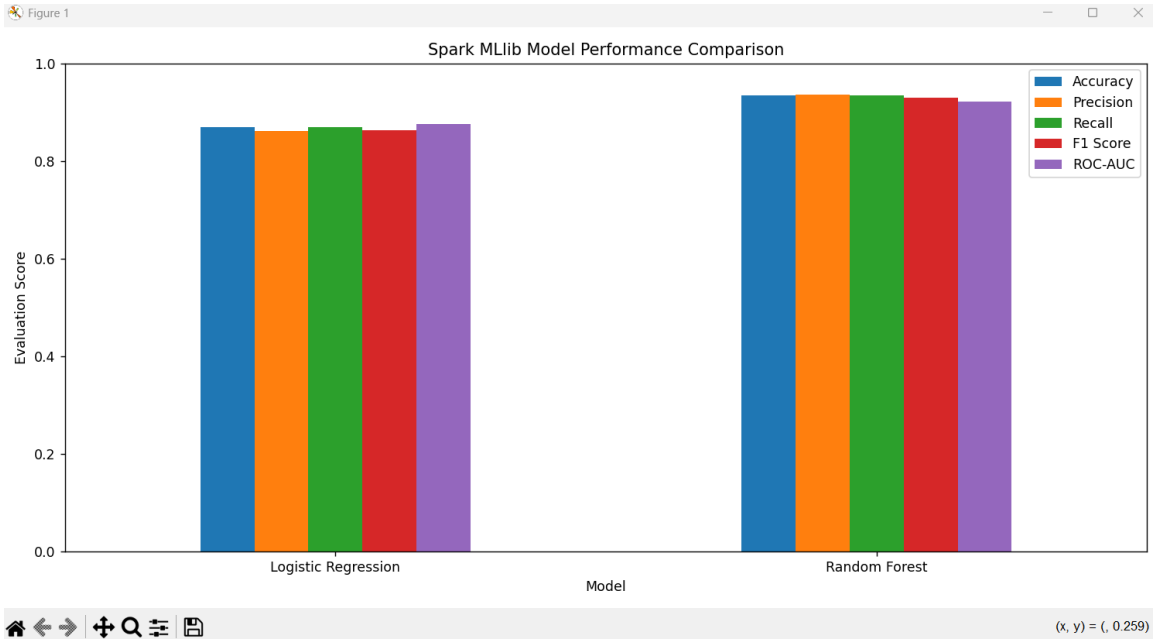


Chart 5: Random Forest Confusion Matrix

The confusion matrix shows the number of correct and incorrect predictions made by the Random Forest model. The model correctly classified 5,084 non-default loans and 940 default loans.

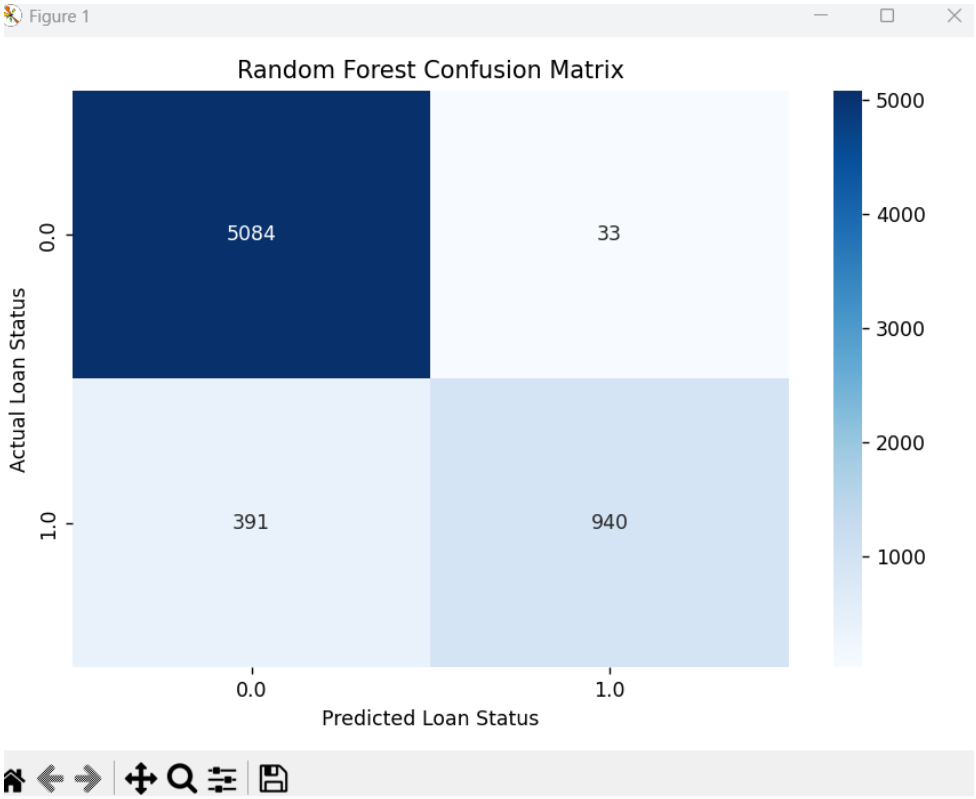
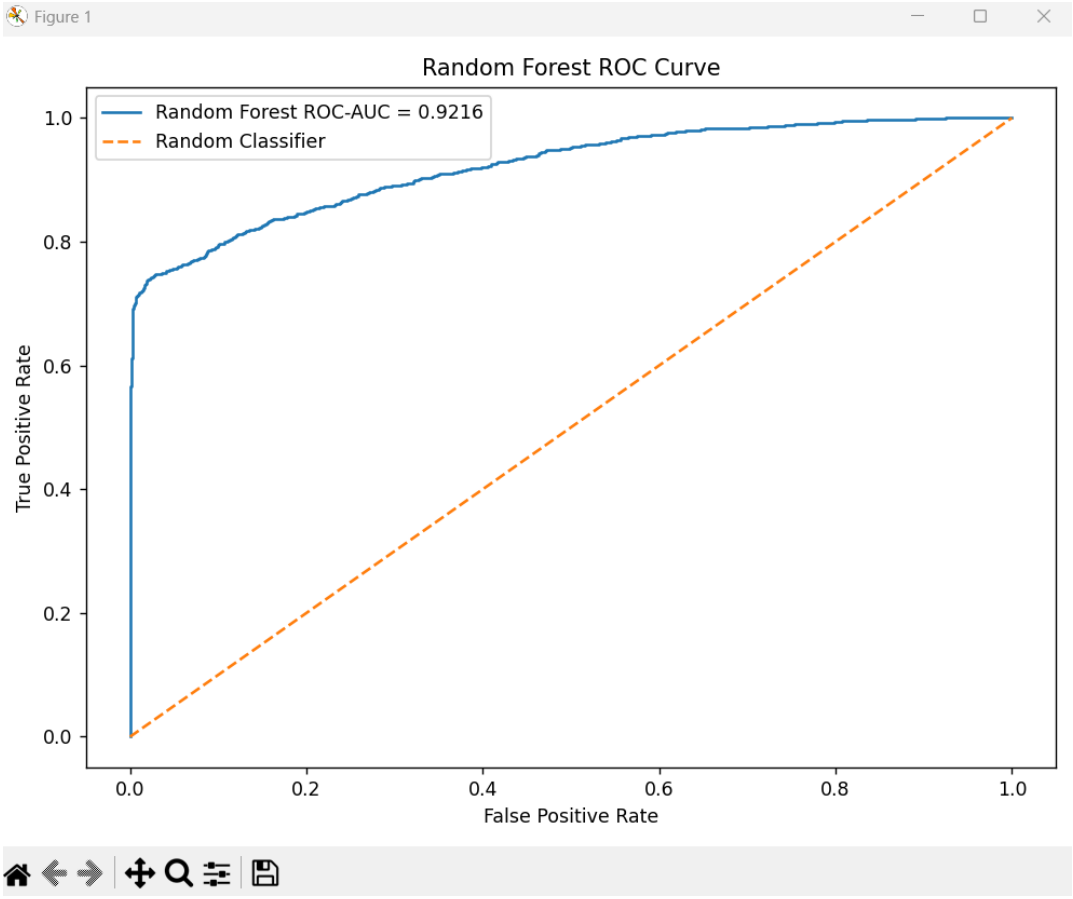


Chart 6: Random Forest ROC Curve

The ROC curve represents the classification performance of the Random Forest model. The model achieved a ROC-AUC score of 0.9216, indicating good ability to distinguish between default and non-default loans.



10. Result Analysis

The project analyzes the credit risk dataset using Apache Spark MLlib and implements two machine learning classification algorithms: Logistic Regression and Random Forest.

Dataset Analysis

The original dataset contains 32,581 records and 12 attributes. Approximately 78.18% of records belong to the non-default category, while 21.82% belong to the default category.

This indicates that the dataset contains more non-default than default loan records.

Model Performance Analysis

Both classification models are evaluated using three metrics:

- **Accuracy:** Measures the proportion of correctly classified loan records.
- **F1 Score:** Measures the balance between precision and recall.
- **ROC-AUC:** Measures the ability of the models to distinguish between default and non-default loans.

Evaluation Metric	Logistic Regression	Random Forest

Accuracy	To be filled	To be filled
F1 Score	To be filled	To be filled
ROC-AUC	To be filled	To be filled

The actual evaluation scores must be entered after executing the program.

Visualization Analysis

The five charts provide insights into loan status distribution, loan purposes, loan grades, applicant income, and model performance.

The model comparison chart helps identify which algorithm performs better on the testing dataset. Since the dataset contains an imbalance between default and non-default records, F1 Score and ROC-AUC are important alongside Accuracy.

The Random Forest prediction results are saved in CSV format for further analysis.

11. Conclusion

The project demonstrates the application of cluster computing technologies and machine learning techniques in financial risk analysis.

Apache Spark and PySpark are used for data loading, preprocessing, and transformation, while Spark MLlib is used to implement Logistic Regression and Random Forest classification models. The dataset is analyzed using Pandas and Matplotlib to generate five visualizations representing loan characteristics and model performance.

Both machine learning models are evaluated using Accuracy, F1 Score, and ROC-AUC to compare their prediction capabilities. The project also demonstrates feature engineering, data visualization, and prediction storage using Spark-based processing.

Overall, the project provides a practical approach to analyzing credit risk and predicting loan defaults. It highlights the importance of Apache Spark MLlib in developing scalable machine learning workflows and supporting data-driven financial risk assessment.

12. References

1. Kaggle, Credit Risk Dataset: https://www.kaggle.com/datasets/laotse/credit-risk-dataset

2. Apache Spark Documentation: https://spark.apache.org/docs/latest/

3. PySpark ML Documentation:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html
4. Pandas Documentation: https://pandas.pydata.org/docs/
5. Matplotlib Documentation: https://matplotlib.org/stable/